INSTITUTO HARDWARE BR

EMBARCATEH – Campinas, SP

MANUAL TÉCNICO

Sistema de Check-In Inteligente

para Clínicas Médicas

Plataforma: Raspberry Pi Pico 2W (Microcontrolador RP2350/RP2040)
Metodologia: Método Cugnasca de Engenharia de Sistemas Críticos
Versão: 1.1
Responsável Técnico: Adriana Rocha Castro de Paula e Luana Menezes
Data: Fevereiro de 2026
Contato: suporte@hardwarebr.edu.br

0. Controle de Versões e Identidade Técnica

Este documento é a única fonte de verdade para operação, manutenção e desenvolvimento do Sistema de Check-In Inteligente. Toda atualização deve ser registrada na tabela abaixo antes de ser distribuída.

Versão	Data	Autor	Descrição das Alterações
1.0	01/02/2026	Adriana / Luana	Lançamento inicial do protótipo funcional.
1.1	18/02/2026	Adriana / Luana	Padronização visual, guia de instalação completo, HAL, seções de ambiente de desenvolvimento, cadastro biométrico, configuração GM67, protocolo R307S e parâmetros FreeRTOS.

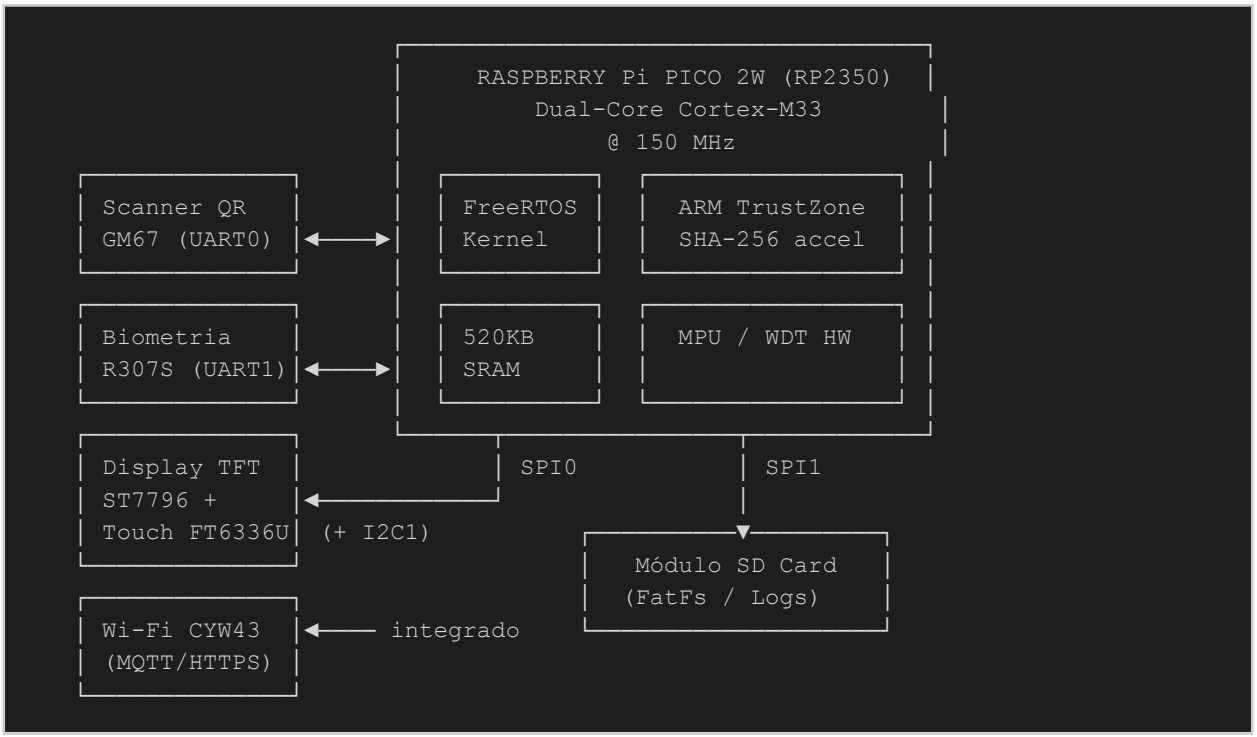
PARTE I — VISÃO GERAL DO SISTEMA

1.1 Introdução e Finalidade

O Sistema de Check-In Inteligente é um totem de atendimento autônomo que automatiza a identificação de pacientes em clínicas médicas, combinando hardware embarcado de alto desempenho com arquitetura de software robusta. O resultado é uma solução de baixo custo, alta disponibilidade e conformidade nativa com a LGPD (Lei 13.709/2018), desenvolvida e validada sob o Método Cugnasca de Engenharia de Sistemas Críticos.

1.2 Diagrama de Blocos do Sistema

A arquitetura opera em topologia estrela, com o RP2350 como núcleo central:



> **i** O isolamento lógico entre periféricos de entrada (sensores) e saída (display) é garantido pelo software. Uma falha em um módulo (ex: biometria danificada) não paralisa o restante do sistema — o check-in por QR Code continua operando.

1.3 Objetivos de Desempenho e Disponibilidade

- **Tempo total de check-in:** inferior a 5 segundos após identificação
- **Taxa de atualização do display:** superior a 30 FPS
- **Watchdog Timer:** reinicialização automática em até 500 ms
- **Heap livre mínimo:** 20% do pool de 192 KB em operação contínua
- **FAR biométrica (R307S):** inferior a 0,001%
- **Disponibilidade offline:** operação completa com sincronização posterior via SD Card

PARTE II — AMBIENTE DE DESENVOLVIMENTO

Esta parte descreve todo o ambiente necessário para compilar, gravar e depurar o firmware do zero. Um técnico sem experiência prévia com RP2350 deve seguir esta sequência antes de qualquer outra etapa.

2.1 Requisitos de Sistema

- **Sistema Operacional:** Windows 10/11 (64-bit), Ubuntu 22.04 LTS ou macOS 13+
- **Git:** versão 2.30 ou superior
- **CMake:** versão 3.17 ou superior
- **GNU Arm Toolchain:** arm-none-eabi-gcc 12.2 ou superior
- **Python:** versão 3.9 ou superior (usado pelo Pico SDK)
- **VS Code:** recomendado, com extensão CMake Tools e Cortex-Debug
- **Driver USB:** Zadig (Windows) para reconhecer o Pico 2W em modo BOOTSEL

2.2 Instalação do Pico SDK (Windows)

Execute os passos abaixo em um terminal PowerShell com privilégios de administrador:

1. Instalar o pacote oficial do Pico SDK para Windows: baixe o instalador em https://github.com/raspberrypi/pico-setup-windows/releases e execute. Ele instala automaticamente o toolchain ARM, CMake, Git e o SDK v2.0+.
2. Após a instalação, abrir o atalho 'Pico - Visual Studio Code' criado na área de trabalho — ele configura as variáveis de ambiente automaticamente.
3. Verificar a instalação: abrir o terminal integrado do VS Code e executar: arm-none-eabi-gcc --version

```
# Saída esperada:
arm-none-eabi-gcc (Arm GNU Toolchain 12.2) 12.2.1 20221205
```

2.3 Instalação do Pico SDK (Linux/Ubuntu)

```
# 1. Instalar dependências do sistema
sudo apt update && sudo apt install -y git cmake gcc-arm-none-eabi \
  libnewlib-arm-none-eabi libstdc++-arm-none-eabi-newlib \
  python3 python3-pip build-essential

# 2. Clonar o Pico SDK (versão compatível com RP2350)
cd ~ && git clone https://github.com/raspberrypi/pico-sdk.git --branch 2.0.0
cd pico-sdk && git submodule update --init

# 3. Configurar variável de ambiente (adicionar ao ~/.bashrc)
echo 'export PICO_SDK_PATH=$HOME/pico-sdk' >> ~/.bashrc
source ~/.bashrc

# 4. Verificar
echo $PICO_SDK_PATH   # deve exibir: /home/usuario/pico-sdk
```

2.4 Clonando o Repositório do Projeto

```
git clone https://github.com/AdryRocha/checkin-inteligente.git
cd checkin-inteligente
git submodule update --init --recursive   # baixa FreeRTOS e LVGL como submodules
```

📌 O projeto usa submódulos Git para FreeRTOS-Kernel e LVGL v9. Sempre execute 'git submodule update --init --recursive' após clonar ou ao fazer pull de atualizações.

2.5 Estrutura de Pastas do Projeto

```
├── CMakeLists.txt
├── pico_sdk_import.cmake
│
├── config/
│   ├── api_config.hpp                  # URL da API, endpoints, credenciais, timeouts
│   ├── display_config.hpp              # Resolução, rotação, SPI speed, buffer LVGL
│   ├── fingerprint_config.hpp          # UART e baudrate do sensor biométrico
│   ├── FreeRTOSConfig.h                # Configuração do kernel FreeRTOS
│   ├── hw_config.c                     # Configuração do SD Card (FatFS)
│   ├── lv_conf.h                       # Configuração do LVGL
│   ├── lvgl_tick_custom.h              # Tick customizado do LVGL para FreeRTOS
│   ├── lwipopts.h                      # Configuração do lwIP (TCP/IP stack)
│   ├── mongoose_config.h               # Configuração do Mongoose HTTP
│   ├── pin_config.hpp                  # Mapeamento de todos os GPIOs
│   ├── qr_code_reader_config.hpp       # UART e baudrate do scanner QR
│   ├── sntp_callbacks.h                # Callbacks de sincronização NTP
│   └── wifi_config.hpp                 # SSID, senha e país do WiFi
│
├── src/
│   ├── main.cpp                        # Entry point: init drivers → create tasks →
scheduler
│   │
│   ├── core/
│   │   ├── state_machine.hpp           # Máquina de estados (singleton, 11 estados)
│   │   └── state_machine.cpp
│   │
│   ├── hal/
│   │   ├── interfaces/
│   │   │   ├── hal_spi_interface.hpp   # Interface abstrata SPI
│   │   │   ├── hal_i2c_interface.hpp   # Interface abstrata I2C
│   │   │   ├── hal_uart_interface.hpp  # Interface abstrata UART
│   │   │   └── hal_wifi_interface.hpp  # Interface abstrata WiFi
│   │   └── rp2040/
│   │       ├── hal_spi_rp2040.hpp/.cpp
│   │       ├── hal_i2c_rp2040.hpp/.cpp
│   │       ├── hal_uart_rp2040.hpp/.cpp
│   │       └── hal_wifi_rp2040.hpp/.cpp
│   │
│   ├── drivers/
│   │   ├── display/
│   │   │   ├── interface/display_interface.hpp
│   │   │   └── st7796/st7796_driver.hpp/.cpp
│   │   ├── touch/
│   │   │   ├── interface/touch_interface.hpp
```

```
│   │   │   └── ft6336u/ft6336u_driver.hpp/.cpp
│   │   ├── qr_code_reader/
│   │   │   ├── interface/qr_interface.hpp
│   │   │   └── gm67/gm67_driver.hpp/.cpp
│   │   └── fingerprint/
│   │       ├── interface/fingerprint_interface.hpp
│   │       └── r307s/r307s_driver.hpp/.cpp
│   │
│   ├── adapters/lvgl/
│   │   ├── lvgl_display_adapter.hpp/.cpp
│   │   └── lvgl_touch_adapter.hpp/.cpp
│   │
│   ├── services/
│   │   ├── appointment_service.hpp/.cpp     # Validação de QR e agendamentos
│   │   ├── data_storage_service.hpp/.cpp    # Leitura/escrita no SD Card
│   │   ├── fingerprint_service.hpp/.cpp     # Cadastro e verificação biométrica
│   │   ├── network_sync_service.hpp/.cpp    # Cliente HTTP (Mongoose)
│   │   └── time_service.hpp/.cpp            # NTP → RTC, data/hora
│   │
│   ├── tasks/
│   │   ├── task_wifi_time_init.hpp/.cpp     # Conexão WiFi + sincronização NTP
│   │   ├── task_lvgl.hpp/.cpp               # Loop LVGL + gerência de telas
│   │   ├── task_qr_reader.hpp/.cpp          # Leitura contínua do scanner QR
│   │   ├── task_qr_validator.hpp/.cpp       # Validação e orquestração do check-in
│   │   └── task_network.hpp/.cpp            # Download de agendamentos e templates
│   │
│   ├── ui/screens/
│   │   ├── welcome_screen.hpp/.cpp          # Tela principal (idle)
│   │   ├── init_status_screen.hpp/.cpp      # Tela de boot (WiFi, NTP, SD)
│   │   ├── appointment_screen.hpp/.cpp      # Check-in confirmado
│   │   ├── error_screen.hpp/.cpp            # Erro recuperável
│   │   ├── error_critical_screen.hpp/.cpp   # Erro crítico + countdown reboot
│   │   ├── fingerprint_operation_screen.hpp/.cpp  # Verificação/cadastro digital
│   │   └── fingerprint_upload_screen.hpp/.cpp     # Upload de template
│   │
│   └── tests/code_examples/
│       ├── README.md                        # Instruções dos demos
│       ├── lvgl_rtos_demo.cpp               # Demo display + touch
│       ├── qr_code_reader_demo.cpp          # Demo scanner QR
│       ├── fingerprint_demo.cpp             # Demo sensor biométrico
│       ├── sdcard_fatfs_demo.cpp            # Demo SD Card
│       └── wifi_demo.cpp                    # Demo WiFi
│
├── lib/
│   ├── FreeRTOS-Kernel/                     # Submodule
│   └── no-OS-FatFS/                         # Submodule
│
└── docs/
    ├── dev/                                 # Notas de desenvolvimento por periférico
    └── diagrams/
        ├── hardware_diagram.md
        ├── software_architecture_layers_diagram.md
        ├── class_diagrams.md
        ├── sequence_diagrams.md
        └── state_machine_diagram.md
```

⚠ NUNCA defina números de GPIO diretamente nos arquivos de driver. Use exclusivamente as definições do arquivo src/config/board_config.h. Isso é um requisito da Metodologia Cugnasca para portabilidade e manutenibilidade.

2.6 Compilando o Firmware

```
# A partir da pasta raiz do projeto:
mkdir build && cd build

# Configurar o CMake — FLAG PICO_BOARD OBRIGATÓRIA
cmake .. -DPICO_BOARD=pico2_w -DPICO_SDK_PATH=$PICO_SDK_PATH

# Compilar (use -j$(nproc) para paralelizar)
make -j$(nproc)

# Arquivo gerado para gravação:
ls build/*.uf2   # ex: checkin_inteligente.uf2
```

⚠ A flag -DPICO_BOARD=pico2_w é OBRIGATÓRIA. Sem ela, o driver do chip wireless CYW43 não é vinculado e todas as funções de rede ficam inoperantes após o flash — o erro não aparece em tempo de compilação.

2.7 Gravando o Firmware (Flash)

4. Conectar o Pico 2W ao PC via cabo USB enquanto mantém pressionado o botão BOOTSEL na placa.
5. Soltar o botão BOOTSEL. O sistema operacional montará uma unidade de armazenamento virtual chamada 'RP2350'.
6. Copiar (arrastar) o arquivo .uf2 gerado para essa unidade.
7. A unidade será desmontada automaticamente e o Pico reiniciará com o novo firmware.
8. Verificar: o LED verde da placa piscará 3 vezes indicando boot bem-sucedido.

```
# Alternativa via linha de comando (Linux/macOS com picotool instalado):
picotool load build/checkin_inteligente.uf2 --force
picotool reboot
```

2.8 Depuração em Tempo Real

Conectar via USB Serial (115200 baud) para visualizar logs das tarefas em tempo real:

```
# Linux/macOS:
screen /dev/ttyACM0 115200

# Windows (PowerShell):
# Usar PuTTY ou Tera Term → porta COM (verificar Gerenciador de Dispositivos)
# Baudrate: 115200 | Data bits: 8 | Stop: 1 | Paridade: nenhuma

# Exemplo de saída esperada no boot:
[BOOT] Sistema de Check-In Inteligente v1.1
[BOOT] SD Card OK — FatFs montado
[BOOT] WiFi: conectando a 'NomeDaRede'...
```

```
[BOOT] WiFi: IP 192.168.1.105 obtido
[BOOT] Watchdog armado — timeout: 500ms
[UI]   Tela de boas-vindas exibida
```

PARTE III — HARDWARE

3.1 Especificações do Microcontrolador RP2350

- **CPU:** Dual-Core ARM Cortex-M33 @ 150 MHz com FPU e DSP
- **SRAM:** 520 KB on-chip, dividida em bancos para acesso DMA paralelo
- **Flash:** 4 MB QSPI externa (código e dados estáticos)
- **Segurança:** ARM TrustZone, Secure Boot, OTP, acelerador SHA-256 por hardware
- **Alimentação:** 5V via USB-C | Consumo médio: 250 mA | Pico: 500 mA
- **Custo:** R$ 127,98 (Ref. NF 24/11/2025)

> ⚠ O Scanner GM67 e o backlight do display DEVEM ser alimentados pelo pino VBUS (5V) — nunca pelo regulador interno de 3,3V — para evitar brown-outs e corrupção de dados no cartão SD.

3.2 Lista de Materiais (BOM)

#	Componente	Interface	Função	Custo Ref.
1	Raspberry Pi Pico 2W (RP2350)	—	Processamento e Controle RTOS	R$ 127,98
2	Display LCD TFT 4.0" (ST7796S)	SPI	Interface Humano-Máquina	R$ 110,00
3	Sensor Biométrico R307S	UART	Autenticação por Impressão Digital	R$ 113,00
4	Scanner QR Code GM67	UART	Leitura de Agendamentos 1D/2D	R$ 150,00
5	Módulo Micro SD Card	SPI	Armazenamento de Logs e Cache	R$ 25,00
6	Fonte USB-C 5V/2A	Power	Alimentação Estabilizada	R$ 45,00
7	Jumpers, resistores 4k7, caps 100nF/10µF	—	Conexões e desacoplamento	~ R$ 15,00
	TOTAL ESTIMADO			R$ 585,98

3.3 Mapeamento de Pinagem (GPIO)

Use exclusivamente as definições de board_config.h. Nunca use valores fixos no código dos drivers.

Periférico	Função	Pino Periférico	GPIO RP2350	Observação Técnica
Display ST7796	SPI Clock	SCK	GP18	SPI0. Freq. < 30 MHz.
Display ST7796	Data Out	MOSI	GP19	Linha de vídeo (alta velocidade).
Display ST7796	Chip Select	CS	GP17	Seleção manual via software.
Display ST7796	Data/Cmd	DC	GP20	Distingue dados de comandos.

Periférico	Função	Pino Periférico	GPIO RP2350	Observação Técnica
Display ST7796	Reset	RST	GP21	Reset de hardware do controlador.
Display ST7796	Backlight	BL	GP22	Controle de brilho via PWM.
Touch FT6336U	I2C Data	SDA	GP11	I2C1. Pull-up 4k7 obrigatório.
Touch FT6336U	I2C Clock	SCL	GP10	I2C1. Modo Fast (400 kHz).
Scanner GM67	UART RX	TX	GP1	UART0. Cruzar TX→RX.
Scanner GM67	UART TX	RX	GP0	UART0. Baudrate: 9600 bps.
Biometria R307S	UART RX	TX	GP8	UART1. Baudrate fixo: 57600 bps.
Biometria R307S	UART TX	RX	GP9	UART1. Protocolo binário c/ Checksum.
SD Card	SPI MISO	DO	GP12	SPI1. Leitura de logs.
SD Card	SPI MOSI	DI	GP15	SPI1. Escrita de logs LGPD.
SD Card	SPI Clock	SCK	GP14	SPI1. Sincronização.
SD Card	Chip Select	CS	GP13	SPI1 CSn dedicado.

```
// src/config/board_config.h — exemplo de definições corretas
#pragma once

// Display ST7796 — SPI0
#define PIN_DISPLAY_SCK   18
#define PIN_DISPLAY_MOSI  19
#define PIN_DISPLAY_CS    17
#define PIN_DISPLAY_DC    20
#define PIN_DISPLAY_RST   21
#define PIN_DISPLAY_BL    22

// Touch FT6336U — I2C1
#define PIN_TOUCH_SDA     11
#define PIN_TOUCH_SCL     10

// Scanner GM67 — UART0
#define PIN_QR_TX         0
#define PIN_QR_RX         1
#define QR_BAUDRATE       9600

// Biometria R307S — UART1
#define PIN_BIO_TX        8
#define PIN_BIO_RX        9
#define BIO_BAUDRATE      57600

// SD Card — SPI1
#define PIN_SD_SCK        14
#define PIN_SD_MOSI       15
#define PIN_SD_MISO       12
#define PIN_SD_CS         13
```

3.4 Alimentação e Gerenciamento de Energia

- **Tensão de entrada:** 5V DC via USB-C ou pinos VSYS
- **Corrente nominal:** 200–300 mA
- **Pico de corrente:** até 500 mA (transmissão Wi-Fi + display ativo simultaneamente)
- **Desacoplamento:** capacitores 100 nF (HF) e 10 µF (bulk) distribuídos no barramento
- **Proteção brown-out:** monitoramento interno de tensão com gravação segura antes do reset

PARTE IV — CONFIGURAÇÃO DOS PERIFÉRICOS

4.1 Configuração do Scanner QR GM67 para UART

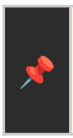
O GM67 vem configurado de fábrica no modo USB-HID (teclado virtual). Para usar com o RP2350 via UART, é obrigatório executar o procedimento de configuração uma única vez antes de montar o totem.

Parâmetros UART do GM67

Parâmetro	Configuração GM67	Observação
Baudrate padrão	9600 bps	Padrão de fábrica. Configurável via comando serial.
Data bits	8	Padrão UART.
Stop bits	1	Padrão UART.
Paridade	Nenhuma	Sem paridade.
Modo de saída	UART (TTL 3,3V)	Deve ser configurado; padrão de fábrica é USB-HID.
Terminador de pacote	\r\n (0x0D 0x0A)	Firmware lê até encontrar \r ou \n.
Timeout de leitura	200 ms	Descarte pacotes sem terminador após 200 ms.

Procedimento de Configuração (via PC, uma vez por módulo)

9. Conectar o GM67 ao PC via cabo USB.
10. Abrir qualquer editor de texto simples (Bloco de Notas).
11. Com o cursor posicionado no editor, escanear o QR Code 'Enable UART Mode' do manual do fabricante GM67. O módulo emitirá um bipe de confirmação.
12. Em seguida, escanear o QR Code 'Set Baud Rate 9600' do manual do fabricante.
13. Desconectar do USB e conectar ao RP2350 conforme a pinagem (GP0/GP1). O módulo agora opera em UART 9600 bps permanentemente (salvo em memória não volátil interna).



Os QR Codes de configuração estão disponíveis no manual oficial do GM67 (arquivo GM67_Barcode.pdf). Guardar uma cópia impressa deste manual junto ao estoque de reposição de módulos.

Inicialização no Firmware

```
// Inicialização da UART0 para o GM67 (em hal/gm67_driver.cpp)
uart_init(uart0, QR_BAUDRATE);            // 9600 bps
gpio_set_function(PIN_QR_TX, GPIO_FUNC_UART);
gpio_set_function(PIN_QR_RX, GPIO_FUNC_UART);
uart_set_format(uart0, 8, 1, UART_PARITY_NONE);
uart_set_fifo_enabled(uart0, true);
```

```
// Configurar interrupção — NUNCA usar busy-wait em produção
uart_set_irq_enables(uart0, true, false);  // RX IRQ habilitado
irq_set_exclusive_handler(UART0_IRQ, gm67_rx_handler);
irq_set_enabled(UART0_IRQ, true);
```

4.2 Configuração do Sensor Biométrico R307S

Protocolo de Pacotes do R307S

O R307S usa um protocolo binário proprietário. O firmware verifica o Checksum de cada pacote; pacotes corrompidos são descartados e uma retransmissão é solicitada automaticamente.

Byte(s)	Campo	Valor / Descrição
0–1	Header	0xEF 0x01 — identificador fixo do fabricante.
2–5	Address	0xFF 0xFF 0xFF 0xFF — endereço de broadcast (padrão).
6	Package ID	0x01 = Comando | 0x02 = Dados | 0x07 = Confirmação (ACK).
7–8	Package Length	Comprimento do campo Data + Checksum (big-endian).
9 … N-2	Data / Instruction	Código do comando (ex: 0x01 = GetImage, 0x02 = Img2Tz).
N-1 … N	Checksum	Soma de Package ID + Package Length + Data (big-endian, 2 bytes).

Principais Comandos Utilizados

Código (Hex)	Comando	Descrição
0x01	GetImage	Captura imagem da impressão digital do sensor óptico.
0x02	Img2Tz(1)	Converte imagem capturada em template — armazena no buffer 1.
0x02	Img2Tz(2)	Converte segunda captura — armazena no buffer 2 (para enroll).
0x05	Search	Busca template no banco interno (matching 1:N). Retorna ID + score.
0x06	RegModel	Combina buffers 1 e 2 para criar template final (uso no cadastro).
0x06	StoreChar	Armazena template gerado na memória flash interna do módulo.
0x0D	DeleteChar	Remove template pelo ID — usado na exclusão LGPD.
0x0E	Empty	Apaga TODOS os templates (usar no factory_reset).

Inicialização no Firmware

```
// Inicialização da UART1 para o R307S (em hal/r307s_driver.cpp)
uart_init(uart1, BIO_BAUDRATE);           // 57600 bps — fixo de fábrica
gpio_set_function(PIN_BIO_TX, GPIO_FUNC_UART);
gpio_set_function(PIN_BIO_RX, GPIO_FUNC_UART);
uart_set_format(uart1, 8, 1, UART_PARITY_NONE);
uart_set_fifo_enabled(uart1, true);

// Interrupção RX — mesma abordagem do GM67
uart_set_irq_enables(uart1, true, false);
irq_set_exclusive_handler(UART1_IRQ, r307s_rx_handler);
irq_set_enabled(UART1_IRQ, true);
```

4.3 Procedimento de Cadastro Biométrico (Enroll)

O cadastro de um novo usuário no sensor R307S exige capturar a digital duas vezes para gerar um template confiável. Este procedimento é executado pelo técnico via menu administrativo da interface LVGL ou via script de bancada.

> ⚠ O cadastro biométrico deve ser realizado SOMENTE após o consentimento explícito do paciente, conforme exigido pela LGPD. O menu de enroll só é acessível com credenciais de administrador.

Fluxo de Enroll (6 etapas)

14. Enviar comando GetImage → sensor captura primeira imagem da digital.
15. Verificar ACK (confirmação byte = 0x00). Se ACK = 0x02 (dedo não detectado), repetir a etapa 1.
16. Enviar comando Img2Tz(1) → converte imagem para template no buffer 1.
17. Solicitar ao usuário que levante e reposicione o dedo. Repetir GetImage e Img2Tz(2) para o buffer 2.
18. Enviar comando RegModel → sensor combina os dois buffers e valida a qualidade do template.
19. Enviar comando StoreChar(ID) → template é salvo na memória flash do R307S com o ID do paciente. Confirmar com ACK 0x00.

```
// Pseudocódigo do fluxo de enroll (task_sensors.cpp)
bool enroll_user(uint16_t user_id) {
    if (r307s_get_image() != ACK_OK) return false;     // 1ª captura
    if (r307s_img2tz(1)    != ACK_OK) return false;    // → buffer 1
    vTaskDelay(pdMS_TO_TICKS(2000));                   // usuário reposiciona
    if (r307s_get_image() != ACK_OK) return false;     // 2ª captura
    if (r307s_img2tz(2)    != ACK_OK) return false;    // → buffer 2
    if (r307s_reg_model()  != ACK_OK) return false;    // combina buffers
    return r307s_store(user_id) == ACK_OK;             // grava com ID
}
```

PARTE V — FIRMWARE E SOFTWARE

5.1 Arquitetura em Camadas

- **HAL (Hardware Abstraction Layer):** drivers de baixo nível: hal_wifi_rp2350.cpp, st7796_driver.cpp, r307s_driver.cpp, gm67_driver.cpp
- **Serviços:** logger.cpp, data_storage.cpp (FatFs), lgpd_compliance.cpp (AES, hashing)
- **Tarefas FreeRTOS:** TaskUI, TaskSensors, TaskWiFi, Safety Monitor — concorrência gerenciada pelo kernel
- **Aplicação (FSM):** Máquina de Estados Finita que governa o fluxo completo de check-in

📌 Flag obrigatória no CMakeLists.txt: set(PICO_BOARD pico2_w). Bibliotecas críticas: FreeRTOS-Kernel, lvgl (v9), no-OS-FatFS, pico-sdk v2.0+.

5.2 Parâmetros do FreeRTOS (FreeRTOSConfig.h)

Parâmetro	Valor	Justificativa
configCPU_CLOCK_HZ	150000000UL	Clock do RP2350 em 150 MHz.
configTICK_RATE_HZ	1000	1 ms por tick — resolução adequada para timeouts biométricos.
configMAX_PRIORITIES	5	Safety(4), UI(3), Sensors(2), WiFi(1), Idle(0).
configTOTAL_HEAP_SIZE	(192*1024)	192 KB reservados. Mínimo de 20% livre em operação.
configMAX_SYSCALL_INTERRUPT_PRIORITY	(5 << 4)	Shift de 4 bits para NVIC do Cortex-M33 (4 bits de prioridade).
configUSE_MUTEXES	1	Obrigatório para spi_mutex e fs_mutex.
configUSE_RECURSIVE_MUTEXES	1	Necessário para operações FatFs aninhadas.
configENABLE_TRUSTZONE	0	TrustZone desabilitado na v1.1. Reservado para implementação futura.
configUSE_TASK_NOTIFICATIONS	1	Comunicação leve entre ISR e tarefas.

```
// FreeRTOSConfig.h — trecho dos parâmetros críticos
#define configCPU_CLOCK_HZ                    150000000UL
#define configTICK_RATE_HZ                    1000
#define configMAX_PRIORITIES                  5
#define configTOTAL_HEAP_SIZE                 (192 * 1024)
#define configMAX_SYSCALL_INTERRUPT_PRIORITY  (5 << 4)  // Cortex-M33: shift 4 bits
#define configUSE_MUTEXES                     1
#define configUSE_RECURSIVE_MUTEXES           1
#define configUSE_TASK_NOTIFICATIONS          1
#define configENABLE_TRUSTZONE                0   // v1.1: desabilitado
#define configCHECK_FOR_STACK_OVERFLOW        2   // modo paranóico de detecção
#define configUSE_MALLOC_FAILED_HOOK          1   // hook para heap esgotado
```

5.3 Tarefas FreeRTOS e Prioridades

Tarefa	Prioridade	Stack	Responsabilidade
Safety Monitor Task	4 (Máx)	4 KB	Alimenta o WDT a cada 100 ms. Se qualquer tarefa crítica travar, o WDT expira em 500 ms e reinicia o sistema automaticamente.
TaskUI	3 (Alta)	16 KB	Gerencia LVGL v9 e renderização SPI. Stack generosa para evitar overflow durante renderização de fontes e imagens.
TaskSensors	2 (Média)	8 KB	Monitora filas de mensagens das UARTs (QR e biometria). Decodifica pacotes e envia eventos estruturados para a FSM.
TaskWiFi	1 (Baixa)	8 KB	Mantém conexão via CYW43, envia logs via MQTT/HTTPS e sincroniza horário via NTP de forma assíncrona.

5.4 Primitivas de Sincronização

- **fsm_event_queue:** canal principal de eventos entre tarefas (EVENT_QR_READ, EVENT_FINGER_MATCH, EVENT_TIMEOUT)
- **spi_mutex:** protege o barramento SPI compartilhado entre TaskUI (display) e DataStorage (SD Card)
- **fs_mutex:** garante atomicidade das operações de escrita/leitura no FatFs
- **wifi_status_sem:** sinaliza à TaskUI quando a conexão Wi-Fi está disponível (atualiza ícone)

```
// Criação das primitivas — em main.cpp, antes de vTaskStartScheduler()
QueueHandle_t   fsm_event_queue  = xQueueCreate(10, sizeof(fsm_event_t));
SemaphoreHandle_t spi_mutex      = xSemaphoreCreateMutex();
SemaphoreHandle_t fs_mutex       = xSemaphoreCreateRecursiveMutex();
SemaphoreHandle_t wifi_status_sem = xSemaphoreCreateBinary();

// Uso correto do spi_mutex em qualquer driver SPI:
if (xSemaphoreTake(spi_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {
    // acesso exclusivo ao barramento SPI
    spi_write_blocking(spi0, data, len);
    xSemaphoreGive(spi_mutex);
} else {
    // timeout — logar erro e retornar falha
    logger_error('SPI mutex timeout em task_ui');
}
```

5.5 Máquina de Estados Finita (FSM)

A FSM governa todas as transições do sistema, impedindo estados indefinidos ou coleta de dados sem autorização. Cada transição possui um temporizador de guarda — se o evento esperado não ocorrer no prazo, o sistema retorna ao estado seguro anterior.

```
Estados da FSM (fsm_checkin.cpp):
```

```
[IDLE]
  |── toque detectado ──────────────────────► [CONSENTIMENTO]
      |── 'ACEITAR' pressionado ──────────► [IDENTIFICACAO]
      |── 'RECUSAR' ou timeout 30s ─────────► [IDLE]
            |── QR Code lido (EVENT_QR_READ) ──► [VALIDACAO]
            |── Digital OK (EVENT_FINGER_OK) ──► [VALIDACAO]
            |── timeout 60s ──────────────────► [IDLE]
                  |── usuário encontrado ──────► [CONFIRMACAO]
                  |── não encontrado ──────────► [ERRO]
                        |── exibe resultado 5s ─► [IDLE]
                        └── TaskWiFi envia log ─► (assíncrono)
```

5.6 Inicialização (setup_minimal)

```
// main.cpp — ordem obrigatória de inicialização
void setup_minimal() {
    // 1. Configurar clock para 150 MHz
    set_sys_clock_khz(150000, true);

    // 2. UART de debug (logs via USB Serial, 115200 baud)
    stdio_usb_init();

    // 3. Armar o Watchdog Timer (500 ms)
    watchdog_enable(500, true);  // pause_on_debug=true

    // 4. Inicializar SPI1 e montar FatFs — FALHA CRÍTICA se não montar
    if (!data_storage_init()) {
        // LED pisca em loop — indica falha no SD Card
        while(true) { gpio_put(LED_PIN, !gpio_get(LED_PIN)); sleep_ms(200); }
    }

    // 5. Inicializar drivers de periféricos
    st7796_init();
    ft6336u_init();
    r307s_init();
    gm67_init();

    // 6. Criar primitivas do FreeRTOS
    fsm_event_queue  = xQueueCreate(10, sizeof(fsm_event_t));
    spi_mutex        = xSemaphoreCreateMutex();
    fs_mutex         = xSemaphoreCreateRecursiveMutex();

    // 7. Criar tarefas
    xTaskCreate(task_ui,       'UI',      16384, NULL, 3, NULL);
    xTaskCreate(task_sensors,  'Sensors', 8192,  NULL, 2, NULL);
    xTaskCreate(task_wifi,     'WiFi',    8192,  NULL, 1, NULL);
    xTaskCreate(task_watchdog, 'WDT',     4096,  NULL, 4, NULL);

    // 8. Transferir controle para o FreeRTOS
    vTaskStartScheduler();  // nunca retorna
}
```

PARTE VI — CONFORMIDADE COM A LGPD

6.1 Princípios Aplicados

- **Minimização de dados:** coletados apenas os dados necessários para o check-in
- **Consentimento explícito:** firmware bloqueia fisicamente os sensores até confirmação. Inatividade de 30 s retorna ao IDLE sem coleta.
- **Anonimização biométrica:** DSP interno do R307S converte imagem em template matemático. Hash AES aplicado antes de qualquer persistência — reconstrução da digital é tecnicamente impossível.
- **Criptografia em repouso:** volume do SD Card protegido por AES simétrico
- **Segurança em trânsito:** HTTPS ou MQTT com TLS, aproveitando acelerador SHA-256 do RP2350

6.2 Estrutura do Log de Auditoria

📋 Formato de cada registro: TIMESTAMP | DEVICE_ID | EVENT_TYPE | USER_HASH_SHA256 | STATUS

Dados de QR Code contendo informações pessoais são processados apenas em memória volátil e descartados após validação, sem persistência em disco.

6.3 Exclusão de Dados (Direito à Remoção)

20. Para excluir um usuário específico: acessar menu de administrador → 'Gerenciar Usuários' → 'Excluir' → confirmar ID. O firmware executa o comando DeleteChar(ID) no R307S e remove o hash correspondente do log.
21. Para limpeza total (descarte do equipamento): criar arquivo vazio chamado factory_reset.flag na raiz do SD Card e reiniciar. O firmware detecta o arquivo, executa o comando Empty no R307S, apaga todos os logs criptografados e remove o arquivo de flag.

⚠ O procedimento de factory_reset é irreversível. Fazer backup dos logs de auditoria LGPD antes de executar, se necessário para fins jurídicos.

PARTE VII — INSTALAÇÃO E COMISSIONAMENTO

7.1 Checklist Completo de Instalação

Passo	Atividade	Descrição Técnica	Critério de Sucesso
1	Montagem Física	Fixar totem em local seco e ventilado. Evitar luz solar direta sobre o leitor GM67 (saturação óptica).	Gabinete estável e leitor livre de reflexos.
2	Energização	Conectar fonte USB-C 5V/2A. Verificar conexão do pino VBUS para backlight e scanner QR.	LED de Power aceso. Backlight ativo sem cintilação.
3	Setup SD Card	Formatar MicroSD Classe 10 em FAT32. Criar arquivo wifi_config.txt na raiz com SSID= e PASS=.	Arquivo salvo sem extensões ocultas.
4	Inserção SD	Com sistema desligado, inserir cartão no slot SPI1. Nunca remover com sistema energizado.	Slot travado e cartão firmemente posicionado.
5	Flash Firmware	Conectar USB ao PC segurando BOOTSEL. Arrastar .uf2 para unidade virtual RP2350.	Dispositivo reinicia e LED verde pisca 3 vezes.
6	Configuração Rede	Sistema lê wifi_config.txt e tenta conexão. Status no ícone Wi-Fi da interface LVGL.	Ícone Wi-Fi ativo na barra de status.
7	Teste Biométrico	Acessar menu técnico e realizar leitura de teste. R307S deve acender luz azul.	Mensagem 'Sensor Ok' e ID reconhecido na tela.
8	Validação LGPD	Iniciar fluxo normal. Verificar se sistema solicita consentimento antes de ativar scanner.	Scanner só emite luz de leitura após 'ACEITAR'.

7.2 Arquivo wifi_config.txt

Criar na raiz do SD Card com exatamente o seguinte formato (sem espaços ao redor do '='):

```
SSID=NomeDaSuaRede
PASS=SuaSenhaWifi
```

⚠ Verificar se o sistema operacional não adicionou extensão oculta (.txt.txt). O firmware não processará o arquivo se o nome estiver incorreto. No Windows, ativar 'Mostrar extensões de arquivo' nas opções de pasta.

7.3 Configuração do GM67 para UART (antes de montar)

Ver seção 4.1 deste manual para o procedimento completo de configuração do GM67 de USB-HID para UART. Este passo é obrigatório e deve ser executado antes de montar o scanner no totem.

PARTE VIII — MANUTENÇÃO E TROUBLESHOOTING

8.1 Logs e Diagnóstico

A análise de falhas sempre começa pelo arquivo system_error.log no SD Card. Para diagnóstico em tempo real, conectar via USB Serial (115.200 baud).

8.2 Tabela de Troubleshooting

Sintoma	Causa Provável	Ação Corretiva
Tela 'Offline'	Perda de conexão Wi-Fi.	Sistema opera em contingência. Dados salvos no SD. Upload automático ao retornar sinal.
Biometria Não Responsiva	Sujidade na janela óptica ou oscilação em 3,3V.	Limpar com microfibra seca. Verificar conectores GP8/GP9. Fallback para QR Code.
Erro de Escrita no SD	Corrupção FatFs.	Substituir cartão por pSLC. Verificar system_error.log.
Display sem imagem	Falha no driver SPI0 ou cabo CS.	Verificar GP17 (CS) e GP18 (SCK). Conferir tensão VBUS no backlight.
LED pisca sem parar	Erro crítico na inicialização.	Verificar system_error.log. Reconectar periféricos UART. Reflash se necessário.
QR Code não lido	GM67 ainda em modo USB-HID.	Executar procedimento de configuração UART do GM67 (seção 7.3).
Watchdog reiniciando em loop	Deadlock em tarefa crítica.	Conectar Serial 115200 baud e inspecionar logs de stack. Verificar spi_mutex.

8.3 Atualização de Firmware

22. Compilar e validar o novo firmware em protótipo de bancada antes de implantar em produção.
23. Verificar flag PICO_BOARD=pico2_w no CMakeLists.txt.
24. Seguir o procedimento de flash da seção 2.7.

📌 Bibliotecas críticas no CMakeLists.txt: FreeRTOS-Kernel, lvgl (v9), no-OS-FatFS. Qualquer atualização de versão dessas bibliotecas deve passar por teste completo de 24h de estabilidade antes de produção.

8.4 Monitoramento de Saúde (Métricas de Produção)

- **Heap livre:** deve permanecer acima de 20% após 1h de uso contínuo. Verificar via log 'HEAP_FREE_KB'
- **Stack watermark:** inspecionar com uxTaskGetStackHighWaterMark() periodicamente
- **Temperatura do enclosure:** manter abaixo de 45°C para garantir operação confiável do RP2350
- **Ciclos de escrita do SD:** substituição preventiva a cada 2 anos por modelo pSLC em uso intensivo
- **Sensor biométrico:** limpeza mensal da janela óptica com microfibra seca

- **Conectores UART:** inspeção trimestral dos cabos GP8/GP9 e GP0/GP1

Glossário Técnico

- **ACK:** Acknowledgement — confirmação de recebimento de pacote
- **AES:** Advanced Encryption Standard — algoritmo de criptografia simétrica
- **BOOTSEL:** Boot Select — botão do Pico 2W para entrar em modo de gravação USB
- **DMA:** Direct Memory Access — transferência de dados sem intervenção da CPU
- **DSP:** Digital Signal Processor — processador de sinais digitais (integrado ao R307S)
- **FAR:** False Acceptance Rate — taxa de falsa aceitação biométrica
- **FatFs:** biblioteca de sistema de arquivos FAT para sistemas embarcados
- **FSM:** Finite State Machine — Máquina de Estados Finita
- **GPIO:** General Purpose Input/Output — pino de propósito geral do microcontrolador
- **HAL:** Hardware Abstraction Layer — camada de abstração de hardware
- **IoMT:** Internet of Medical Things — Internet das Coisas Médicas
- **IRQ:** Interrupt Request — solicitação de interrupção de hardware
- **LGPD:** Lei Geral de Proteção de Dados — Lei 13.709/2018
- **LVGL:** Light and Versatile Graphics Library — biblioteca gráfica para embarcados
- **MPU:** Memory Protection Unit — unidade de proteção de memória do ARM
- **MQTT:** Message Queuing Telemetry Transport — protocolo leve de telemetria IoT
- **NTP:** Network Time Protocol — sincronização de relógio via rede
- **OTA:** Over-the-Air — atualização de firmware por rede
- **pSLC:** pseudo Single Level Cell — tecnologia de SD Card de grau industrial
- **SDK:** Software Development Kit — kit de desenvolvimento de software
- **TrustZone:** tecnologia ARM de isolamento de execução segura no RP2350
- **UF2:** USB Flashing Format — formato de arquivo de firmware para gravação via USB
- **WDT:** Watchdog Timer — temporizador de hardware que reinicia o sistema em caso de travamento

Suporte Técnico: Instituto Hardware BR | suporte@hardwarebr.edu.br